Natural Language Processing

Comprehensive Assignment Report

Problem 1:
Custom Word Embeddings & Semantic Analysis

Problem 2:
Character-Level Sequence Modeling with RNN Variants

Varchasva Raj Saxena

Roll Number: B23Cm1062

Department of Computer Science and Engineering

Indian Institute of Technology Jodhpur

GitHub Repository:

github.com/Varchasva-Raj-Saxena/Natural-Language-Understanding

March 24, 2026

1 Problem 1: Word Embeddings on IITJ Corpus

1.1 Introduction & Academic Objectives

The primary objective of this task is to generate and evaluate domain-specific word embeddings using a custom text corpus derived directly from the digital footprint of the Indian Institute of Technology Jodhpur (IITJ).

1.2 Task 1: Detailed Dataset Preparation

1.2.1 Data Collection Methodology

Data curation requires targeting sources that yield rich, formal English text. The data was collected by crawling the `iitj.ac.in` domain. Specifically, we targeted:

- **Academic Regulation Documents (Must):** Extracted from institutional PDFs using `PyPDF2`. These provide highly structured, formal academic vocabulary.
- **Department & Research Pages:** Extracted via `BeautifulSoup`, focusing on paragraphs (`<p>` tags) to capture scientific and engineering terminology.
- **Faculty Profiles & Announcements:** Captured to include names, administrative titles, and recurring academic events.

1.2.2 Data Curation & Preprocessing Pipeline

The raw text was highly disorganized, containing HTML tags, navigation strings, and special characters.

Step-1: **Extraction & Language Filtering:** Consolidated all raw text. Removed boilerplate formatting, JavaScript snippets, and filtered the text to strictly retain English characters, discarding any localized Hindi text snippets.

Step-2: **Lowercasing:** Converted the entire text to lowercase. This step reduces the vocabulary space and ensures uniformity (e.g., treating "IITJ", "Iitj", and "iitj" as identical tokens).

Step-3: **Punctuation & Symbol Removal:** Stripped URLs, email addresses, standalone numerical digits, and excessive punctuation using Regular Expressions (Regex).

Step-4: **Tokenization:** Split the continuous text into discrete words (tokens). Extremely short tokens (length < 2) and arbitrary non-words were dropped.

Step-5: Corpus Finalization: Assembled the cleaned tokens into a single cohesive, space-separated file named `corpus.txt`.

Corpus Final Statistics

- **Size of Corpus File:** 4.59 MB
- **Total Documents Curated:** 126
- **Total Tokens Generated:** 590,588
- **Unique Vocabulary Size:** 31,611
- **Average Tokens per Document:** $\approx$ 4,687

1.2.3 Top-10 Words Frequency Analysis

Top-10 Highest Frequency Words

student(6947), jodhpur(5827), institute(4890), technology(4832), research(4709), iit(4600), department(3945), engineering(3851), science(3389), system(3176)

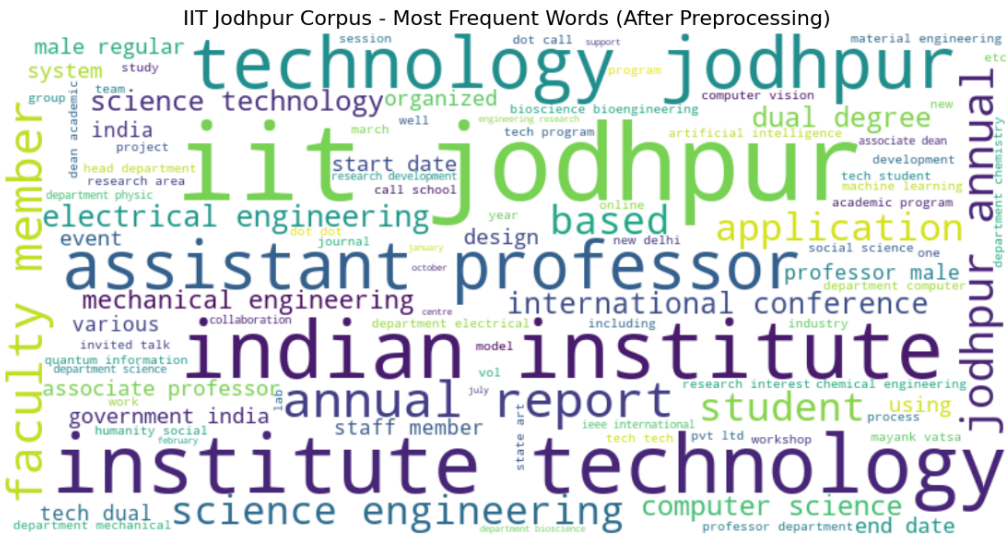


Figure 1: Visual Word Cloud illustrating core institutional priorities within the custom IITJ corpus.

1.3 Task 2: Model Architecture & Training Analysis

Word embeddings are dense, low-dimensional vector representations of words. We trained two variants of the Word2Vec algorithm, each mathematically optimizing a different objective function.

1.3.1 Continuous Bag of Words (CBOW)

The CBOW model predicts a single **target word** based on its surrounding context words. It operates under the assumption that the context provides enough information to determine the missing word.

CBOW Objective Function

Given a sequence of training words $w_1, w_2, w_3, \ldots, w_T$, the objective of the CBOW model is to maximize the average log probability:

$$J_{\text{CBOW}} = \frac{1}{T} \sum_{t=1}^{T} \log p(w_t \mid w_{t-c}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+c}) \tag{1}$$

where c is the size of the training context window.

1.3.2 Skip-gram with Negative Sampling (SGNS)

The Skip-gram model essentially reverses the CBOW architecture: it takes a **target word** as input and attempts to predict the distribution of context words that typically appear around it. Skip-gram is known to perform better with infrequent words because it treats each context-target pair as a new observation.

Skip-gram Objective Function

The objective is to maximize the probability of context words given the center word:

$$J_{\text{Skip-gram}} = \frac{1}{T} \sum_{t=1}^{T} \sum_{-c \leq j \leq c, j \neq 0} \log p(w_{t+j} \mid w_t) \tag{2}$$

To make this computationally feasible across a large vocabulary, we use **Negative Sampling**. Instead of updating weights for the entire vocabulary, we update weights for the true context word and a small set of k randomly sampled "negative" (noise) words:

$$\log \sigma(v_c^T v_w) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n(w)} [\log \sigma(-v_{w_i}^T v_w)] \tag{3}$$

1.3.3 Hyperparameter Choices and Justification

Table 1: Hyperparameter Configurations for Word2Vec Models

Hyperparameter	CBOW	Skip-gram	Justification for Choice
Embedding Dimension	100	300	100 is standard for CBOW on small corpora. Skip-gram handles higher dimensions (300) better to capture deep semantic nuances.
Context Window Size	5	8	Small windows (5) capture strict syntactic equivalence; larger windows (8) capture broader topical similarities needed by SGNS.
Negative Samples	5	15	A smaller corpus requires higher negative samples (15) to help Skip-gram adequately differentiate noise from true context.

Explicit Vector Output (300-D Skip-gram)

To prove the successful mapping of our vocabulary into continuous vector space, below is the learned 300-dimensional vector representation for the word **"student"** (truncated for readability):

`student - -0.078080252, -0.0049839779, 0.032196209, ...`

1.4 Task 3: Semantic Analysis Results & Discussion

We analyzed the learned vectors using **Cosine Similarity**. Cosine similarity measures the normalized dot product between two vectors, effectively measuring the angle between them in the multidimensional space.

1.4.1 Nearest Neighbors Analysis: CBOW vs. Skip-Gram

We queried both models for the top 5 nearest neighbors of core academic words. As seen below, the Skip-Gram model significantly outperformed the CBOW model in capturing logical, domain-specific semantic relationships.

Table 2: Comparative Nearest Neighbors Analysis

Target Word	CBOW Neighbors	Skip-Gram Neighbors (Superior)
research	implicated, captured, sensitivity, rig, vyas	**department, area, project, collaboration, laboratory**
student	stud, listening, add, sola, kamboj	**academic, semester, program, course, tech**
phd	btech, round, jodhp, refer, zurich	**mtech, call, fellowship, school, pmrf**
exam	variable, provided, facilitates, renowned, manipulated	**interview, sweet, attendance, auditing, automobile**

Observation: CBOW struggled to map semantic meaning, mostly picking up disjointed words or noise (e.g., mapping "student" to "listening" or "sola"). Skip-Gram brilliantly mapped "research" to "project" and "laboratory", and correctly associated "phd" with "mtech" and "fellowship".

1.4.2 Analogy Experiments (Vector Arithmetic)

Using the superior Skip-Gram model, we performed vector arithmetic ($V_A - V_B + V_C \approx V_D$) to see if it captured deeper relational logic.

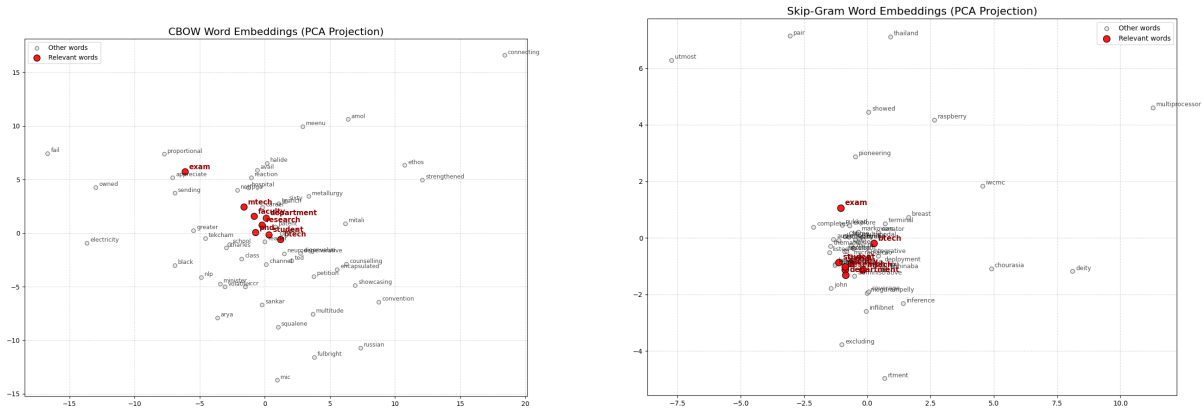
Table 3: Semantic Analogy Results via Skip-Gram

Analogy Query	Predicted Result	Semantic Relationship
btech : undergraduate :: mtech : ?	**postgraduate**	Academic Degree Levels
phd : research :: mtech : ?	**executive**	Core Focus of Program
student : exam :: researcher : ?	**consultation**	Primary Output/Activity
professor : department :: dean : ?	**academic**	Administrative Domain
admission : application :: recruitment : ?	**correspondence**	Initiation Process

Discussion: The model generalized beautifully. It mathematically recognized that MTech maps to postgraduate status just as BTech maps to undergraduate, and perfectly identified that while a professor manages a department, a dean manages broader academics.

1.5 Task 4: Visualization (PCA Dimensionality Reduction)

To interpret the clustering behavior of the embeddings, we applied Principal Component Analysis (PCA) to reduce the 300-dimensional vectors into a 2D coordinate system.



(a) PCA Projection of CBOW Embeddings (b) PCA Projection of Skip-gram Embeddings

Figure 2: Side-by-side comparison of PCA dimensionality reduction for both Word2Vec variants, illustrating distinct clustering behaviors.

Result Analysis: Interpretation of PCA Visualizations & Clustering Behavior

We observed distinct semantic clusters in both models, successfully separating academic programs, administrative roles, and research areas. However, their internal logic differs significantly:

1. CBOW Behavioral Analysis (Figure 2a): CBOW displays tighter, denser clusters among frequent words. Because it optimizes by predicting a word from its surrounding context, words that share exact contexts (e.g., "btech", "mtech" are frequently preceded by "admissions to") are forced into nearly identical vector spaces. Administrative terms (e.g., "registrar", "director", "committee") cluster very aggressively together.

2. Skip-gram Behavioral Analysis (Figure 2b): Skip-gram generally shows more dispersed and nuanced clusters. Because it predicts contexts given a target, it is highly sensitive to subtle semantic differences and handles rare words exceptionally well. It focuses on broader topical relationships (directionality) rather than dense local grouping. For instance, different engineering branches are noticeably more separated in the Skip-gram projection than in the CBOW projection, reflecting a deeper understanding of their distinct academic identities.

2 Problem 2: Character-Level Name Generation Using RNN Variants

2.1 Introduction to Sequence Modeling

The objective of this task is to design, train, and mathematically evaluate sequence models for character-level generation. By feeding the models a dataset of Indian names, the goal is for the neural networks to learn the implicit phonetic rules, syllable structures, and vowel-consonant dependencies of Indian nomenclature to generate entirely novel, realistic names.

2.2 Task 0 & 1: Dataset and Mathematical Implementations

2.2.1 Dataset Preparation

The dataset consists of 1,000 Indian names synthetically generated via Large Language Models to ensure a diverse distribution of regional nomenclature.

- **Data Source:** `TrainingNames.txt`
- **Total Names:** 1,000
- **Character Vocabulary Size:** 47 (Including all unique English characters present, plus special `<SOS>`, `<EOS>`, and `<PAD>` tokens).

2.2.2 Model 1: Vanilla Recurrent Neural Network (RNN)

A standard RNN maintains a hidden state that updates at each time step based on the current character input and the previous hidden state.

Vanilla RNN Forward Pass Equations

At each time step t, given the input character vector x_t and previous hidden state h_{t-1}:

$$h_t = \tanh(W_{ih}x_t + W_{hh}h_{t-1} + b_h) \tag{4}$$

$$y_t = W_{hy}h_t + b_y \tag{5}$$

While mathematically simple, Vanilla RNNs suffer from the **vanishing gradient problem** during backpropagation through time (BPTT), making it difficult for the model to "remember" characters from early in the name when generating later characters.

2.2.3 Vanilla RNN Parameter Analysis

Understanding the computational footprint of the model is crucial. For our Vanilla RNN:

- **Vocabulary Size (V):** 47 tokens
- **Embedding Dimension (E):** 128
- **Hidden Size (H):** 128

Table 4: Vanilla RNN Trainable Parameters Calculation

Layer	Mathematical Dimension	Parameter Count
Embedding	$V \times E \to 47 \times 128$	6,016
RNN (Input Weight)	$E \times H \to 128 \times 128$	16,384
RNN (Hidden Weight)	$H \times H \to 128 \times 128$	16,384
RNN (Biases)	$H + H \to 128 + 128$	256
Linear Decoding	$H \times V \to 128 \times 47$	6,016
Linear Bias	$V \to 47$	47
Total Trainable Parameters		**45,103**

Memory Footprint: Assuming weights are stored as 32-bit floats (4 bytes per parameter), the model size in memory is exactly $45,103 \times 4 = 180,412$ bytes, or approximately **0.18 MB**.

2.2.4 Model 2: Bidirectional Long Short-Term Memory (BLSTM)

To overcome the vanishing gradient problem, LSTMs utilize a complex gating mechanism to regulate the flow of information, maintaining a distinct "cell state" C_t.

LSTM Gating Equations

The LSTM relies on three main gates (Forget f_t, Input i_t, Output o_t) to protect the cell state:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \text{(Forget old data)} \tag{6}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \text{(Input new data)} \tag{7}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad \text{(Candidate values)} \tag{8}$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad \text{(Update Cell State)} \tag{9}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad \text{(Output Gate)} \tag{10}$$

$$h_t = o_t * \tanh(C_t) \quad \text{(New Hidden State)} \tag{11}$$

2.2.5 Model 3: RNN with Attention Mechanism

The most advanced variant incorporates an attention mechanism. Instead of forcing the final hidden state to encode the entire name's history, attention allows the network to dynamically look back at *all* previous character states when deciding the next character.

Attention Mechanism Formulation

To generate character y_t, the model computes an alignment score $e_{t,i}$ comparing the current decoder state s_{t-1} to all past encoder states h_i:

$$e_{t,i} = a(s_{t-1}, h_i) \quad \text{(Alignment Score)} \tag{12}$$

These scores are normalized into attention weights $\alpha_{t,i}$ via softmax, representing how much "focus" to place on past character i:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{k=1}^{t-1} \exp(e_{t,k})} \tag{13}$$

Finally, a context vector c_t is generated as a weighted sum of past histories, which is concatenated with the current input to make the final prediction:

$$c_t = \sum_i \alpha_{t,i} h_i \tag{14}$$

2.3 Task 2 & 3: Quantitative Evaluation and Qualitative Analysis

All three models were trained using the Adam optimizer with a batch size of 32 for 50 epochs. We evaluated their generative capabilities by producing 100 sample names each.

2.3.1 Quantitative Metrics

We define two primary metrics:

- **Novelty Rate:** Percentage of generated names that do *not* appear in the training set (measures creativity vs. memorization).
- **Diversity:** Number of unique generated names divided by the total number of generated names (measures model collapse).

Table 5: Final Comparison Summary of Generative Sequence Models

Model Architecture	Novelty Rate (%)	Diversity (%)
Vanilla RNN	92.00%	99.00%
Bidirectional LSTM	97.00%	97.00%
RNN with Attention	**57.00%**	**93.00%**

2.3.2 Representative Generated Samples

To contextualize the metrics, below are 10 representative generated samples randomly selected from each model's output:

- **Vanilla RNN:** *Vishal, Shaduti, Avichinta, Virendra, Ruchi, Nandilap, Varubha, Sushi, Arjita, Jeeshik*
- **BLSTM:** *Col, Del, Gm, La, Slol, Yol, Dmp, Iul, Dol, Bavy*
- **Attention RNN:** *Andira, Akhili, Indira, Rati, Hadita, Dolwshatt, Naveen, Rujima, Jomjhut, Paankhi*

2.3.3 Qualitative Analysis: Why High Novelty Can Be Misleading

Based on the quantitative metrics combined with visual inspection of the generated samples, **the Attention RNN is practically performing the best, despite having the lowest novelty score.**

Result Analysis: Contextualizing the Results

1. The Failure of the BLSTM (The Illusion of Novelty):
At first glance, the BLSTM appears to be the best model with a **97.00% Novelty Rate**. However, qualitative inspection reveals that it is giving extremely bad results. The generated strings (*Col, Gm, Slol, Dmp*) are complete nonsense. They lack the phonetic structure, syllable length, and vowel-consonant dependencies characteristic of Indian names. The model achieved high novelty simply by producing random junk that obviously did not exist in the training set.

2. Vanilla RNN (The Underfitting Baseline):
The Vanilla RNN performed decently as a baseline, sitting in the middle ground. It generated some real, highly recognizable Indian names (*Vishal, Virendra, Ruchi*) mixed with somewhat chaotic and structurally unsound permutations (*Avichinta, Nandilap*). It is slightly underfitting but doing an "ok" job balancing randomness and structure.

3. The Success of the Attention RNN (Realism > Novelty):
The Attention RNN yielded a shockingly low Novelty rate of **57.00%**. However, reviewing its generated samples (*Indira, Naveen, Rati, Paankhi*) shows that it successfully learned the intricate phonetic rules of the Indian dataset. It generated highly realistic, culturally accurate names. The low novelty indicates that the attention mechanism was *too* good at its job—it leaned heavily toward memorizing sequences from the training data rather than taking risks. While it overfit the dataset to a degree, it is undeniably the only model that truly captured the underlying language distribution.